ASSIGNMENT

EXAM CENTER SYSTEM

HIGH-LEVEL DESIGN (HLD) & LOW-LEVEL DESIGN (LLD)

Document Version: 1.0
Prepared by: Group – 01
Date: 04 February 2026
Status: Final Design Document

TABLE OF CONTENTS

PART 1: HIGH-LEVEL DESIGN (HLD)

1. INTRODUCTION

1.1 Purpose

The purpose of this High-Level Design (HLD) document is to describe the overall architectural structure and major design decisions of the **Exam Center Assignment System**. This document acts as a bridge between the Software Requirements Specification (SRS) and the actual system implementation.

It provides developers and reviewers with a clear understanding of how the system is organized, how different components interact with each other, and how the system fulfills the functional and non-functional requirements specified in the SRS. The HLD ensures that development proceeds in a structured, maintainable, and scalable manner while strictly adhering to the defined scope.

1.2 Scope

This design document covers the complete architectural design of the Exam Center Assignment System, including:

- System architecture and major components
- Data flow and processing logic
- Multi-threading approach and synchronization strategy
- File-based input and output handling
- Capacity-based exam center assignment logic

The document **does not include** any network-based communication, database usage, authentication mechanisms, or web interfaces, as these are explicitly outside the scope defined in the SRS.

1.3 Definitions

To maintain clarity and consistency, the following key terms are used throughout this document:

- **Candidate:** An individual registered to appear for an examination
- **Exam Center:** A physical location with a fixed capacity where candidates are assigned
- **Mutex:** A synchronization mechanism used to prevent concurrent access to shared resources
- **POSIX Threads:** A standardized threading model used for concurrent execution
- **CLI:** Command Line Interface
- **HLD:** High-Level Design
- **LLD:** Low-Level Design

1.4 Overview

This document is divided into two major sections.
The **High-Level Design (HLD)** section explains the system architecture, components, data flow, and design considerations.
The **Low-Level Design (LLD)** section provides a deeper look into class-level design, module responsibilities, and implementation constraints.

Together, these sections provide a complete design blueprint for the Exam Center Assignment System.

2. GENERAL DESCRIPTION

2.1 Product Perspective

The Exam Center Assignment System is a **standalone, console-based C++ application**. It operates independently without reliance on external systems such as databases, servers, or network services.

The system follows a **layered and modular architecture**, ensuring separation of concerns and ease of maintenance. The major layers of the system include:

1. **Input Processing Layer** – Responsible for reading candidate data from input files
2. **Validation Layer** – Ensures that candidate information is complete and correctly formatted
3. **Assignment Logic Layer** – Assigns candidates to exam centers based on exam type and capacity
4. **Output Generation Layer** – Produces output files for assigned, unassigned, and invalid candidates

This architectural approach improves readability, testability, and scalability of the system.

2.2 Tools Used

The system is developed using standard and widely supported tools and technologies:

- **Programming Language:** C++
- **Compiler:** GNU C++ Compiler (g++).
- **Threading Library:** POSIX Threads (pthread).
- **Data Structures:** C++ Standard Template Library (STL).
- **Operating Systems:** Linux and Windows.
- **Build Tool:** Makefile.
- **Testing Tools:** Valgrind and CPPUnit.

These tools ensure portability, performance, and reliability across platforms.

2.3 General Constraints

The system design is subject to the following constraints:

- The number of exam centers is fixed at four.
- All input and output operations must be file-based.
- The application must be executed using the command line.
- POSIX threads must be used for multi-threading.
- One mutex must be associated with each exam center.
- Object-oriented programming principles must be followed.
- No database or network communication is permitted.

These constraints directly originate from the SRS and are strictly enforced in the design.

2.4 Assumptions

The design assumes that:

- Input files are provided in the correct format.
- The system is executed by an authorized user.
- Required software tools are properly installed.
- The operating system supports POSIX threads.
- Adequate memory and CPU resources are available.

These assumptions allow the system to focus on correctness and performance rather than environmental error handling.

2.5 Special Design Aspects

Several important design aspects enhance the system’s robustness and efficiency:

- Multi-threaded file processing for faster execution.
- Mutex-based synchronization to prevent race conditions.
- Object-oriented class hierarchy for candidate types.
- Strict capacity enforcement for exam centers.
- Modular structure for easier maintenance and extension.

3. SYSTEM DESIGN DETAILS

3.1 Main Design Features

The core design features of the system include:

- Use of object-oriented programming concepts such as inheritance and polymorphism.
- Parallel processing of input files using POSIX threads.
- Thread-safe access to shared exam center resources.
- Efficient file-based input and output handling.
- Comprehensive validation and error handling.
- Dynamic memory allocation with proper cleanup.

These features collectively ensure system correctness, efficiency, and reliability.

3.2 Application Architecture

The application follows a **layered architecture** with clearly defined responsibilities:

- **File Processor:** Reads and parses input files.
- **Candidate Validator:** Validates candidate data.
- **Assignment Manager:** Controls assignment logic.
- **Exam Center Manager:** Manages capacity and assignments.
- **Thread Manager:** Handles thread creation and execution.
- **Output Generator:** Writes results to output files.

Each component interacts with others through well-defined interfaces, ensuring loose coupling.

3.3 Data Flow Description

At a high level, data flows through the system as follows:

1. Input files are read from the command line.
2. Candidate records are parsed and stored.
3. Invalid records are separated during validation.
4. Valid candidates are assigned to exam centers based on exam type.
5. Capacity constraints are checked before assignment.
6. Assigned, unassigned, and invalid candidates are written to output files.

This structured flow ensures predictable and correct system behaviour.

3.4 File Handling

Input Files:
Each input file corresponds to a specific examination type and contains candidate records.

Output Files:
The system generates:

- Exam-wise assigned candidate lists.
- A list of unassigned candidates.
- A list of invalid candidates.
- Optional individual hall ticket files.

All file operations are performed using standard C++ file streams.

3.5 User Interface

The system uses a **Command Line Interface (CLI)**.
Users execute the application by providing input file names as arguments. Output is displayed through console messages and generated files.

This interface ensures simplicity and ease of use.

3.6 Error Handling

The system handles errors gracefully, including:

- File read/write failures.
- Invalid input formats.
- Capacity overflow conditions.
- Thread execution errors.

Clear error messages are displayed to guide the user.

3.7 Performance Considerations

Performance is optimized through:

- Parallel processing using threads.
- Efficient STL container usage.
- Minimal disk I/O operations.
- Controlled memory allocation.

These measures ensure timely processing even with large input files.

3.8 Security Considerations

Security is addressed through:

- Strict input validation.
- OS-level file permission handling.
- Thread-safe access to shared resources.
- Safe memory handling practices.

No authentication is required as per system scope.

3.9 Maintainability, Portability, and Reusability

The modular design ensures:

- Easy maintenance and debugging.
- Portability across supported platforms.
- Reusability of core modules for similar systems.

4. SYSTEM ARCHITECTURE DIAGRAMS

4.1 High-Level System Architecture

This diagram illustrates the layered architecture and the flow of data between core components.

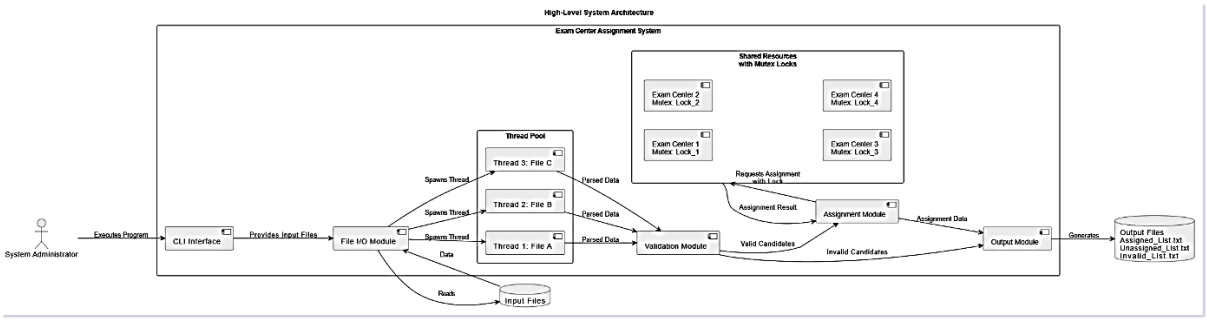


Diagram Explanation:

- The system starts when the administrator runs the program using the command line interface (CLI).
- Input files are read by the File I/O Module.
- Multiple threads are created so that each input file can be processed in parallel.
- Each thread reads candidate details from its assigned file.
- The candidate data is sent to the Validation Module for correctness checking.
- Valid candidate records are forwarded to the Assignment Module.
- Invalid candidate records are separated for reporting.
- The Assignment Module assigns valid candidates to exam centers based on exam type.
- Each exam center is protected by a dedicated mutex to ensure thread-safe access.
- Capacity of exam centers is checked before assigning candidates.

- Candidates exceeding capacity are marked as unassigned.
- The Output Module generates final output files for:
    1. Assigned candidates
    2. Unassigned candidates
    3. Invalid candidates

4.2 System Data Flow Diagram (DFD Level 0)

This diagram shows the end-to-end flow of data through the system's major processes.

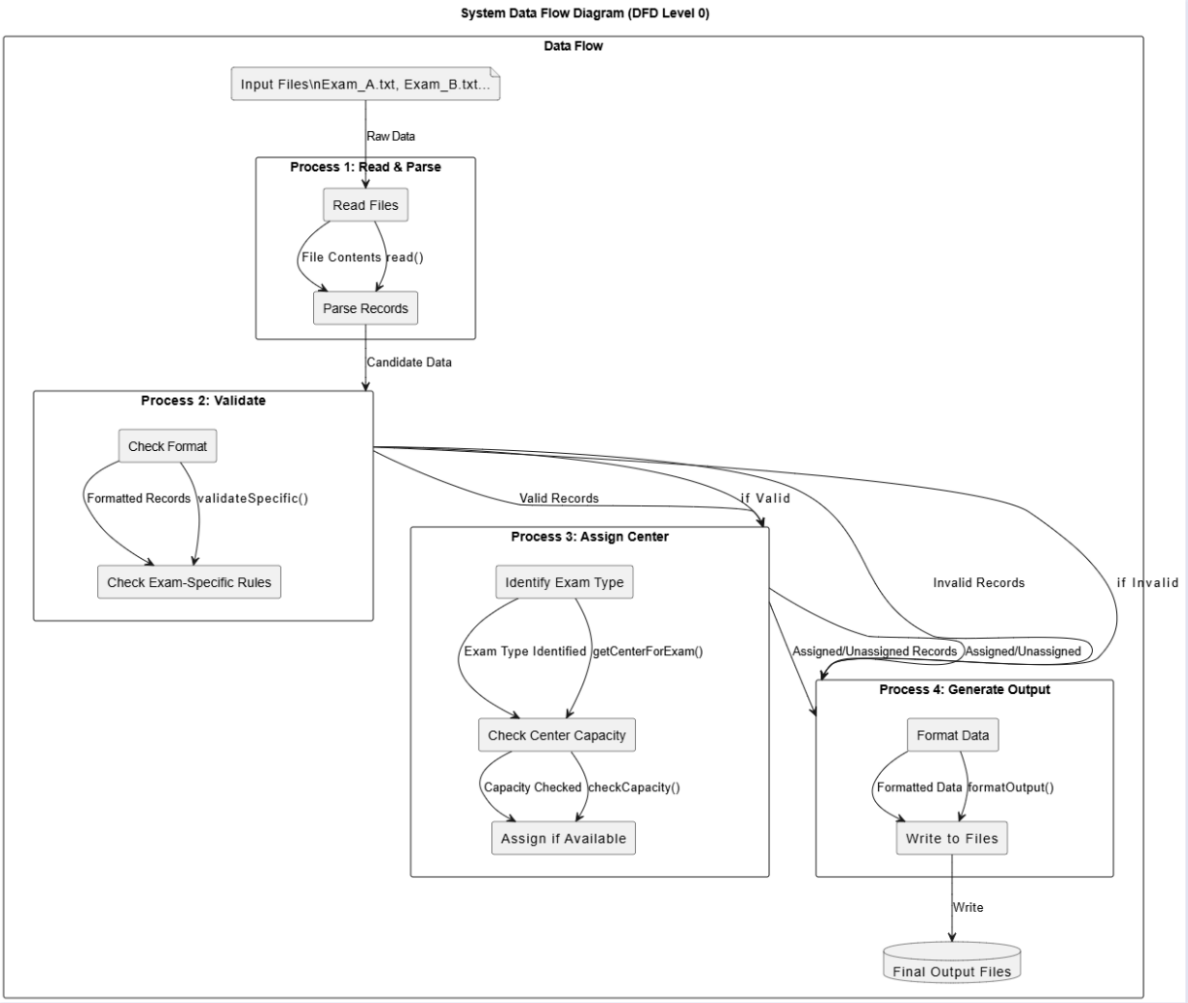


Diagram Explanation:

- Data starts from the input files provided to the system.
- **Process 1** reads the input files and converts raw data into structured candidate records.
- The candidate records are passed to **Process 2** for validation.
- **Process 2** checks each record against format and business rules.
- Invalid records are separated and marked as invalid.
- Valid records are forwarded to **Process 3**.
- **Process 3** performs the exam center assignment logic.
- The system checks the capacity of the appropriate exam center.

- If capacity is available, the candidate is marked as assigned.
- If capacity is full, the candidate is marked as unassigned.
- All records (assigned, unassigned, and invalid) are sent to **Process 4**.
- **Process 4** formats the data and writes it to their respective output files.
- The data flow ends after all output files are successfully generated.

4.3 Data Flow Diagram (DFD) – Level 1

The Level 1 Data Flow Diagram illustrates the major functional processes of the **Exam Center Assignment System** and shows how data flows between external entities, system processes, and data stores.

External Entity

- **System Administrator**
  The administrator executes the application and provides candidate input files through the command line interface.

Major Processes

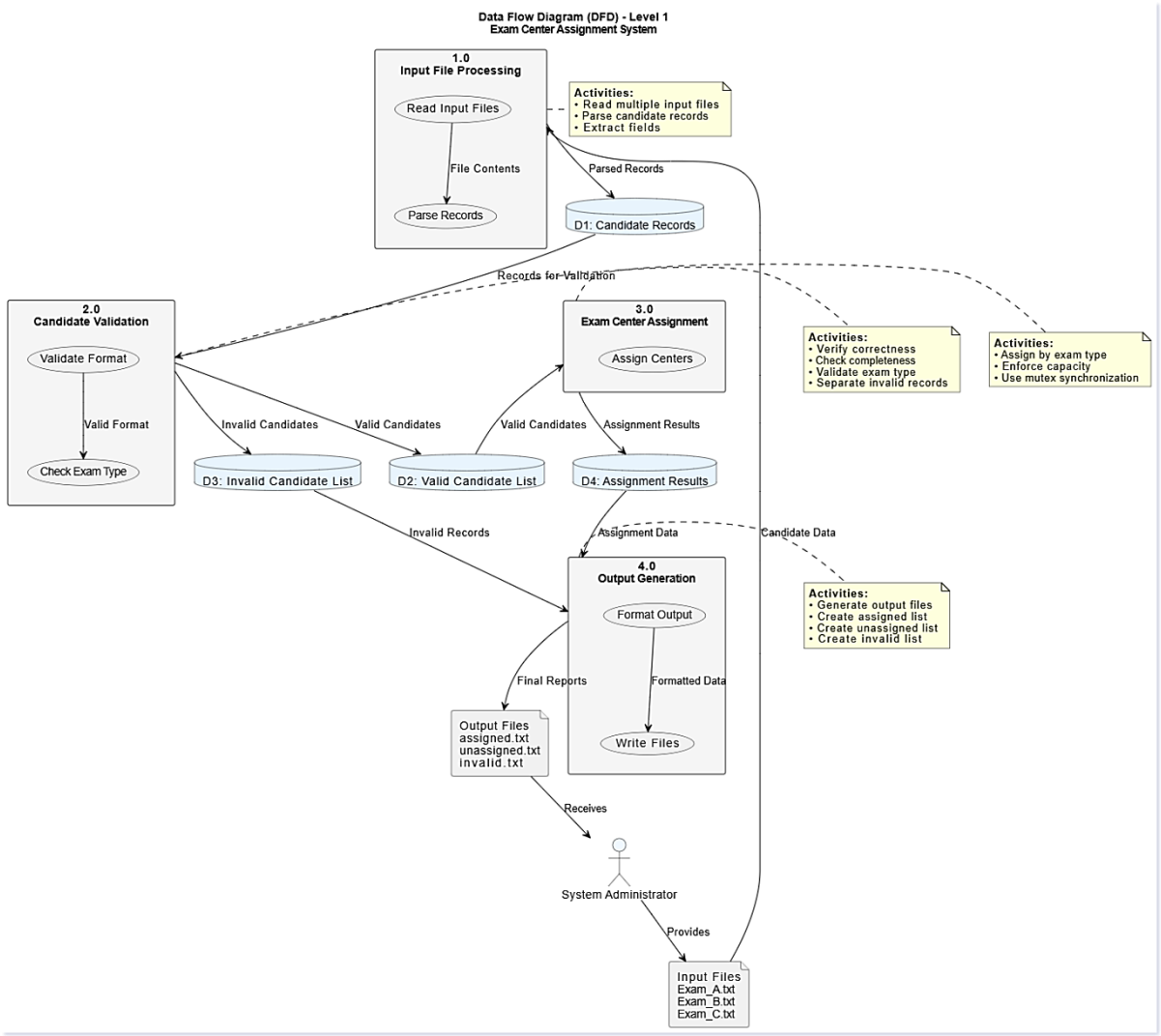
1. **Input File Processing**
   This process reads multiple candidate input files supplied as command-line arguments. It parses each file and extracts individual candidate records for further processing.
2. **Candidate Validation**
   This process verifies candidate records for correctness, completeness, and validity of exam type. Invalid records are separated from valid ones.
3. **Exam Center Assignment**
   This process assigns valid candidates to exam centers based on examination type while strictly enforcing predefined capacity constraints.
4. **Output Generation**
   This process generates output files containing assigned candidates, unassigned candidates due to capacity overflow, and invalid candidate records.

Data Stores

- **D1: Candidate Records** – Temporary storage for parsed candidate data.
- **D2: Valid Candidate List** – Stores validated candidate records.
- **D3: Invalid Candidate List** – Stores invalid candidate records.
- **D4: Assignment Results** – Stores final assignment details.

Data Flow

- Input files → Input File Processing
- Parsed records → Candidate Validation
- Valid candidates → Exam Center Assignment
- Invalid candidates → Output Generation
- Assignment results → Output Generation
- Output files → System Administrator



4.4 Data Flow Diagram (DFD) – Level 2

The Level 2 Data Flow Diagram provides a detailed decomposition of the **Exam Center Assignment** process, showing internal sub-processes involved in allocating candidates to exam centers.

Expanded Process: Exam Center Assignment

1. **Identify Exam Type**
   Determines the examination type of each validated candidate to select the appropriate exam center.
2. **Select Exam Center**
   Chooses the corresponding exam center based on the identified examination type.
3. **Check Exam Center Capacity**
   Verifies available capacity in the selected exam center using mutex-based synchronization to ensure thread safety.
4. **Assign Candidate to Center**
   Assigns the candidate to the exam center if capacity is available and updates the seat count.

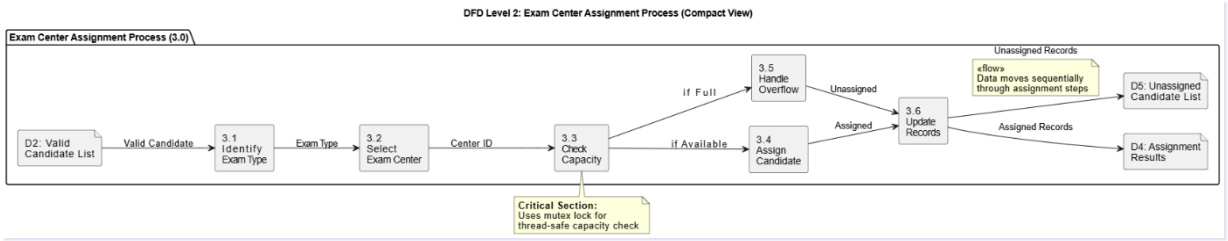
5. **Handle Capacity Overflow**
   Marks the candidate as unassigned if the exam center has reached its maximum capacity.
6. **Update Assignment Records**
   Updates data stores with assigned and unassigned candidate information for final reporting.

Data Stores Used

- **D2: Valid Candidate List.**
- **D4: Assignment Results.**
- **D5: Unassigned Candidate List.**

Data Flow

- Valid candidate → Identify Exam Type
- Exam type → Select Exam Center
- Selected center → Check Capacity
- Capacity available → Assign Candidate
- Capacity full → Handle Overflow
- Assignment status → Update Assignment Records



PART 2: LOW-LEVEL DESIGN (LLD)

5. INTRODUCTION

5.1 Purpose

The Low-Level Design (LLD) document provides detailed implementation-level information required for coding the system. It describes class structures, responsibilities, module interactions, and constraints.

5.2 Document Conventions

- Class names use PascalCase.
- Method and variable names use camelCase.
- Constants use uppercase letters.
- Indentation and formatting follow standard C++ conventions.

5.3 Intended Audience

This document is intended for:

- Software developers.
- Test engineers.
- Code reviewers.
- Academic evaluators.

6. DETAILED SYSTEM DESIGN

6.1 Design Description

The system applies core object-oriented principles:

- **Encapsulation:** Data is accessed through methods.
- **Inheritance:** Candidate types extend a base class.
- **Polymorphism:** Exam-specific validation logic.
- **RAII:** Automatic resource management for mutexes.

6.2 Modular Design

The system is divided into the following modules:

- File I/O Module.
- Validation Module.
- Assignment Module.
- Thread Management Module.
- Reporting Module.
- Utility Module.

Each module has a clear responsibility and minimal dependency on others.

6.3 Class Design Overview

- **Candidate (Abstract Class):** Stores common candidate data
- **Derived Candidate Classes:** Implement exam-specific validation
- **ExamCenter:** Manages capacity and assigned candidates
- **AssignmentManager:** Controls overall assignment logic
- **FileProcessor:** Handles input and output operations
- **ThreadManager:** Manages POSIX threads

6.4 Use Case Description

The primary actor is the **System Administrator**, who:

- Executes the program
- Provides input files
- Reviews generated output

Key use cases include loading files, validating data, assigning centers, generating reports, and exiting the system.

6.5 Design and Implementation Constraints

The system strictly adheres to constraints such as:

- Use of C++ only
- POSIX threads for concurrency
- Fixed number of exam centers
- File-based processing
- Mandatory object-oriented design

6.6 User Interface Design

The CLI supports basic execution and optional flags. Output feedback is provided via console messages and generated files.

6.7 Security Design

Security focuses on correctness and stability through validation, synchronization, and safe memory handling.

7. DETAILED CLASS AND SEQUENCE DIAGRAMS

7.1 Detailed Class Diagram

This diagram details the core classes, their attributes, methods, and relationships.

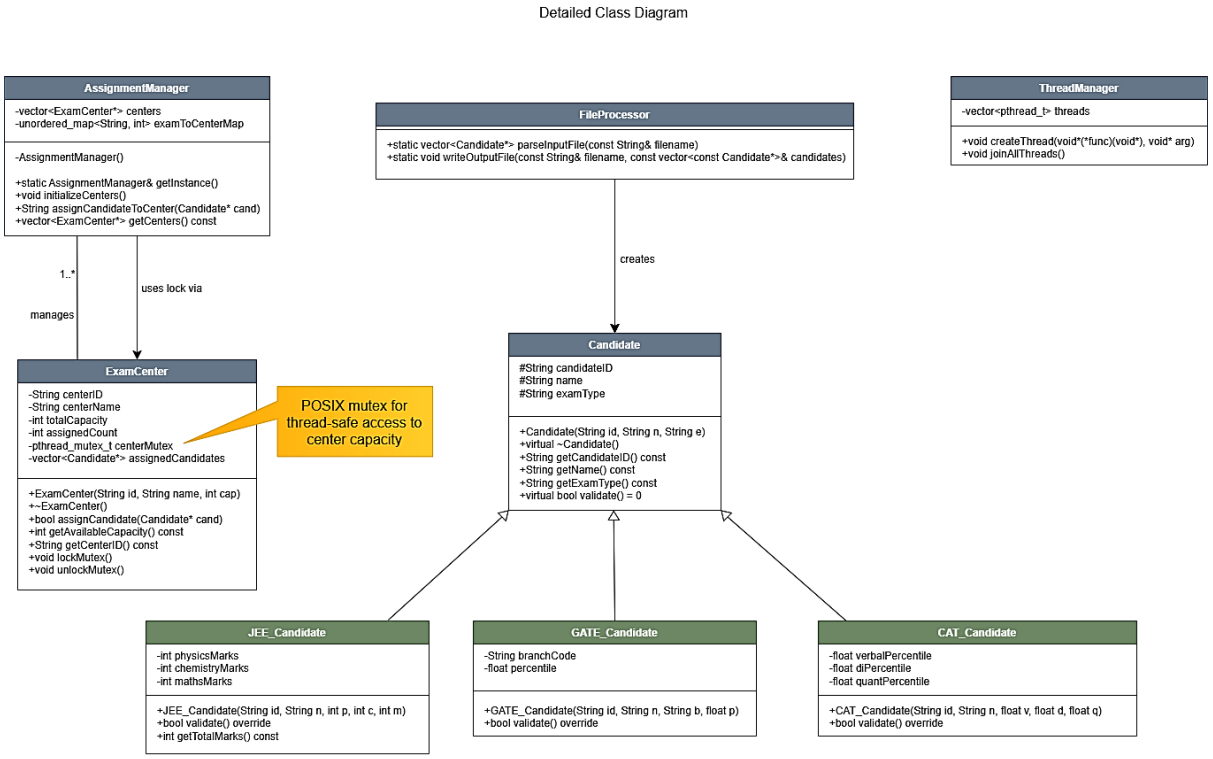


Diagram Explanation:

- The class hierarchy is based on an abstract **Candidate** base class.
- The **Candidate** class contains common attributes shared by all candidates.
- It also defines a pure virtual method validate() for data validation.
- **JEE_Candidate**, **GATE_Candidate**, and **CAT_Candidate** classes inherit from the **Candidate** base class.
- Each derived candidate class provides its own exam-specific validation logic.
- The **ExamCenter** class manages all exam center–related information.
- It maintains the center capacity, a list of assigned candidates, and a mutex lock.
- The **ExamCenter** class provides thread-safe methods such as assignCandidate() for safe candidate assignment.
- The **AssignmentManager** class is implemented using the Singleton design pattern.
- It centrally manages all exam centers and maps exam types to the correct centers.
- The **FileProcessor** class is responsible for reading input files and writing output files.
- The **ThreadManager** class handles thread creation, execution, and lifecycle management.

7.2 Core Assignment Sequence Diagram

This sequence diagram details the interactions between objects during the critical candidate assignment process.

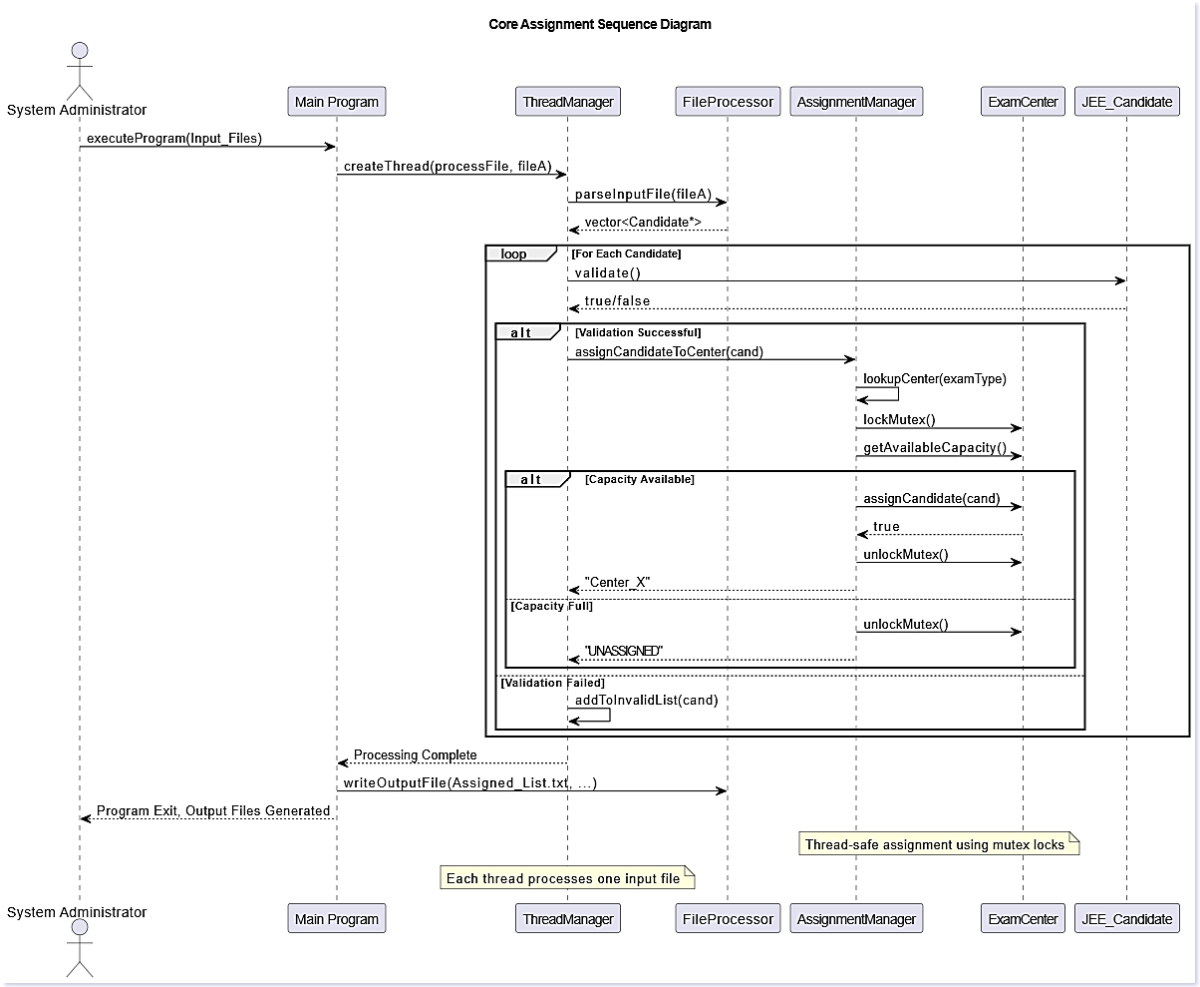


Diagram Explanation:

- The sequence starts when the administrator launches the program.
- The **Main Program** instructs the **ThreadManager** to create a thread for each input file.
- Each thread begins processing its assigned input file.
- The thread uses the **FileProcessor** to read and parse candidate data.
- For every candidate, the validate() method is called.
- If the candidate data is invalid, the candidate is marked as invalid and separated.
- If the candidate is valid, the thread sends an assignment request to the **AssignmentManager** (Singleton).
- The **AssignmentManager** identifies the appropriate **ExamCenter** based on exam type.
- The mutex of the selected exam center is locked to ensure thread safety.
- The system checks whether capacity is available in the exam center.
- If capacity is available:
    1. The candidate is assigned to the exam center.

    2. The mutex is unlocked.
    3. The assigned center ID is returned.
- If the exam center is full:
    1. The mutex is unlocked.
    2. An **UNASSIGNED** status is returned.
- After all threads complete execution, the **Main Program** collects all results.
- The **FileProcessor** is used to generate the final output files for:
    1. Assigned candidates
    2. Unassigned candidates
    3. Invalid candidates